Computer Security 2026 Side channels defenses and Spectre (HW security)

Roberto Guanciale

Making life of attacker harder

Diversification

Counter attacks by making difficult for the attacker to predict the results of their activities

Run-time Defenses: Address Space Randomization

- ASLR randomizes the locations of memory mappings.
- Linux can randomize the stack, shared libraries, other mappings, and heap placement.
- Position Independent Executables (PIE) also support randomization of the main executable.
- Different regions need not share one random displacement.
- Offsets within a loaded image generally remain fixed.
- Address disclosures can undermine protection.
- To disable it: `echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`

Run-time Defenses: Base Stack Randomization

- The OS varies stack placement when creating a new program image.
- The stack pointer tracks the current stack position.
- Randomization makes absolute stack addresses harder to predict.
- Relative offsets within a stack frame may remain predictable.
- Stack placement randomization generally requires no recompilation

Run-time Defenses: Base Stack Randomization

———

```
void f() {
    char [16] pwd;
    char [16] input;
    ...
    gets(input);
}
```

Effective to counter modifications pwd?

Run-time Defenses: Base Stack Randomization

```
void f() {
    char [16] buffer;
    gets(buffer)
}
void nuke() {
}
```

Effective to counter redirection to nuke?

```
void f() {
    char [16] buffer_for_shell;
    gets(buffer_for_shell)
}
```

Effective to counter execution of shell in buffer?

Run-time Defenses: Heap Randomization

- Heap addresses depend on OS mappings and allocator behavior.
  - `ptr = malloc(1024);`
- ASLR varies base adheap and mapping locations.
- Some allocators add randomization to object placement or reuse.
- ASLR does not imply a random order of individual allocations, but that their addresses are difficult to predict
- Randomization does not prevent use-after-free or double-free bugs
- Randomization does not affect iter data structure layout

```
struct {

    char x[10];

    char y[10];

}
```

Run-time Defenses: Global Randomization

- Globals in a PIE executable move with that executable's load address.
- Their relative offsets generally remain fixed.
- Position-independent code and relocations support loading at different bases.
- Randomizing individual globals requires additional transformations.

```
X = MEM[1MB]

===>

offset = MEM[offset-add]

X = MEM[1MB + offset]
```

Run-time Defenses: ASLR

- PIE allows the main executable to load at varying addresses.
- Function and gadget addresses change with the load base.
- An address disclosure (e.g. side channels) may reveal that base.
- ASLR complicates exploitation but does not enforce valid control flow.
- Programs must use location independent code

```
1    MB: jmp 1.2 MB          ==> 2    MB: jmp PC + 0.2 MB

1.2 MB: code                 ==> 2.2  MB: code
```

Artificial software diversity

- forces attackers to target each system individually
  - aims to increase the cost to attackers by randomizing implementation aspects of programs
- without knowledge of a particular program instance, targeted attacks become significantly harder
- Used by malwares (victims) to protect against antiviruses (attacker)
  - Encrypting viruses
  - Polymorphic viruses

What to diversify: Instruction Level

- affects few instructions in a program block
  - Equivalent Instruction Substitution
    - X+X == X*2
  - Equivalent Instruction Sequences
    - X+=1; Z=Y*X == Z=Y*X; X+=1; Z+=Y
  - Instruction Reordering
    - X+1; Y=2 == Y=2; X+=1
  - Register Allocation Randomization
  - Garbage Code Insertion
    - X+=1 == Y+=Y; X+=1; Y/=2
- effective against some code-reuse attacks

What to diversify: Basic Block Level

- Intra-Procedure transformations
  - Basic block reordering
    - A;B;C == jmp A; B; jmp C; A; jmp B; C
  - Opaque Predicate Insertion
    - A == if c then A else A
  - Branch Function Insertion
    - A;l:call f;B A';l':call f;B'
      - f:C;return
    - A;l:call f;B A';l':call f;B'
      - f:C; ret-ptr==l? jmp l; jmp l'
    - A;l:call f 1;B A';l':call f 2;B'
      - f x:C; x==1? jmp l; jmp l'
- Hinder reverse engineering and code-reuse attacks

What to diversify: Function level

- Stack Layout Randomization
  - stack frame padding,
  - stack variable reordering,
  - stack growth reversal, and
  - non-contiguous stack allocation
- Function Parameter Randomization
- Inlining, Outlining, and Splitting
- Control Flow Flattening
  - Replacing direct jumps with indirect ones
  - jmp 0x100 == R1=0x100; jmp R1

When to diversify

- Compile time
  - Each host receives a different program
- Loading time
  - Deferred randomization
  - Each execution is different
  - Program stored on disk is the same
- Run-time
  - Re-randomized while it runs

Be prepared for the worst

Isolation

- Memory safety does not eliminate logic errors or all security vulnerabilities.
- It is difficult to write bug free code
  - https://www.vidarholen.net/contents/wordcount
- The trusted computing base contains the components whose correct behavior is necessary for a security property.
- Reduce this trusted base and give each component only the authority it needs.
- Isolate critical components from faults or compromise in less trusted components.
- Isolation depends on correct enforcement and controlled communication across boundaries.

Isolation: Smart thermostat



- Components
  - Safety controller: reads temperature and decides the heater state.
  - Sensor and heater drivers: access the physical devices.
  - Network service: receives remote temperature requests. (can be compromised)
  - ML optimizer: proposes an energy-saving temperature. (can be compromised)
  - Logger: records events.
- Required control policy
  - At or below 15°C, request heating.
  - At or above 25°C, request no heating.
  - Between these limits, follow a validated comfort preference.
  - Evaluate the policy every second, even when ML or Network Services are compromised

Isolation: Smart thermostat



- Safety limits are fixed trusted configuration
- Use bounded queues, bounded request processing, reserved control-loop CPU time, and non-blocking logging.
- Safety controller
  - Read sensor, command heater via drivers
  - Receive and validate (bounded) target requests, publish status via ML and Network
  - Valid network requests override ML suggestions.
  - Log events
- Drivers
  - Access only their assigned devices;
  - All sensors and actuators: accept commands only from the controller
  - NIC driver: accept commands only from the network service
- Network service
  - Submit target requests; read limited status via safety controller
  - Log user requests
- ML optimizer
  - Read approved temperature history, submit target suggestions via safety controller
  - Log AI decisions
- Logger
  - Receive events; write its own log storage

Isolation: Smart thermostat



- Write heater registers directly
  - Blocked. The network service cannot map device registers or invoke the heater driver’s restricted command interface.
- Request 5°C
  - Rejected by the controller. Validation remains effective even after the network service is compromised.
- Send a million requests
  - Bounded impact. Excess requests are dropped Bounded processing and scheduling isolation preserve the control loop.
- Stop sending requests
  - Control continues. The controller independently requests heating at or below 15°C.
- Repeatedly request 24°C while ML requests 18°C
  - Allowed within the policy. User requests take precedence, so the compromised network service can affect comfort and energy use. Isolation preserves the safety policy, not the authenticity of user intent through a compromised service.
- Modify a log entry
  - Direct modification is blocked. The network service can only append to log storage. However, if not bounded can lead to a DOS

Microkernels

- L4 and Minix is the most famous

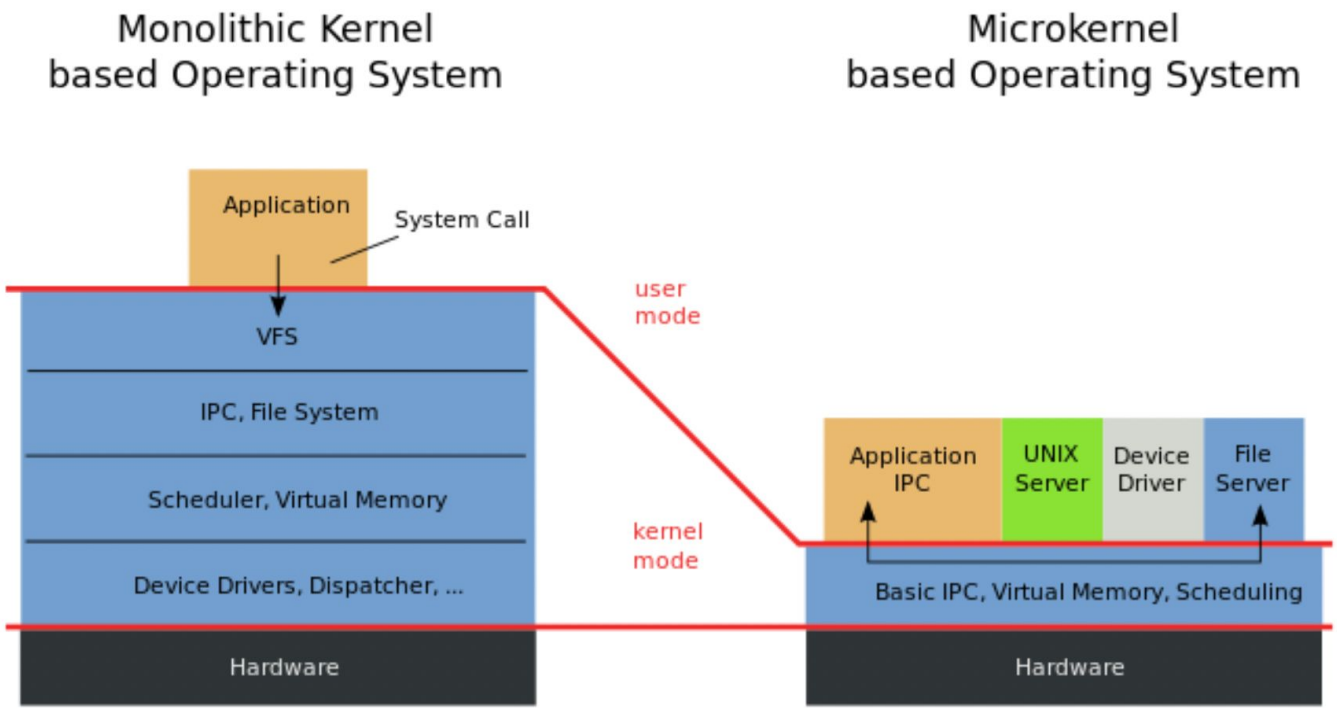
“A concept is tolerated inside the microkernel only if moving it outside the kernel, i.e., permitting competing implementations, would prevent the implementation of the system's required functionality”

- address spaces
- Threads
- Scheduling
- inter-thread communication

Everything else is outside the kernel (e.g. drivers) 15 thousands lines of code

Linux 30 M Loc vs seL4 30 K Loc

Microkernels



Software Fault Isolation

Sandbox non-critical code

Google Chrome Native Client (discontinued)

Modify binary to ensure that overflows can not access critical resources

E.g. everything above 0x01000000 is critical everything below 0x00FFFFFF is non-critical

```
mem[x] := y

=======>

x := x & 0x00FFFFFF

mem[x] := y
```

Software Fault Isolation

Sandbox non-critical code

Google Chrome Native Client (discontinued)

Modify binary to ensure that overflows can not access critical resources

E.g. everything above 0x01000000 is critical everything below 0x00FFFFFF is non-critical

```
mem[x] := y

x+=1

mem[x] := y

=======>

x := x & 0x00FFFFFF

mem[x] := y

x+=1

x := x & 0x00FFFFFF

mem[x] := y
```

Software Fault Isolation

Sandbox non-critical code

Google Chrome Native Client (discontinued)

Modify binary to ensure that overflows can not access critical resources

E.g. everything above 0x01000000 is critical everything below 0x00FFFFFF is non-critical

```
mem[x] := y

x+=1

mem[x] := y

=======>

x := x & 0x00EFFFFF

mem[x] := y

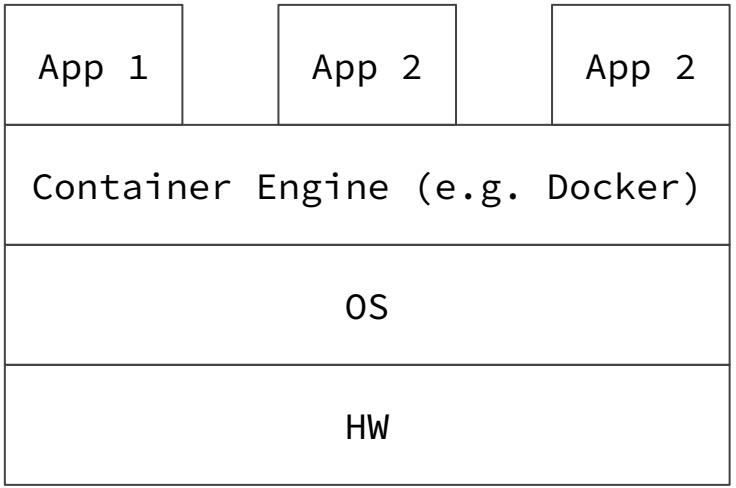
x+=1

mem[x] := y
```

Containers

Virtualise OS

Reuse the same OS



Hypervisors

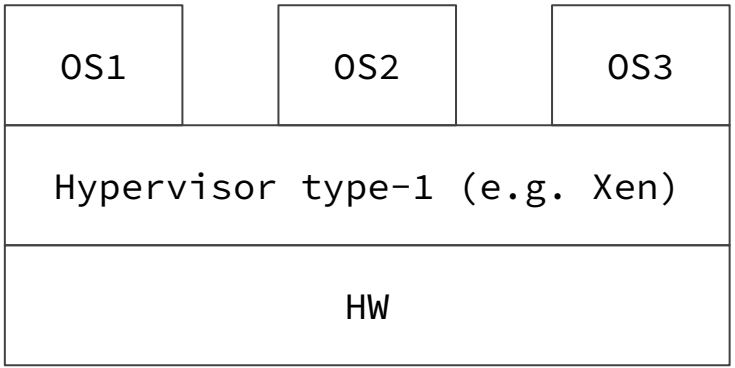
Execute below OS

Isolate complete OSes from each other

Can inspect the behavior of a (possibly) buggy OS

Today hypervisors use HW support:

- Stage 1 MMU configured by OS and maps Virtual Addresses to Intermediate Addresses
- Stage 2 MMU configured by Hypervisor and maps Intermediate Addresses to Physical Addresses

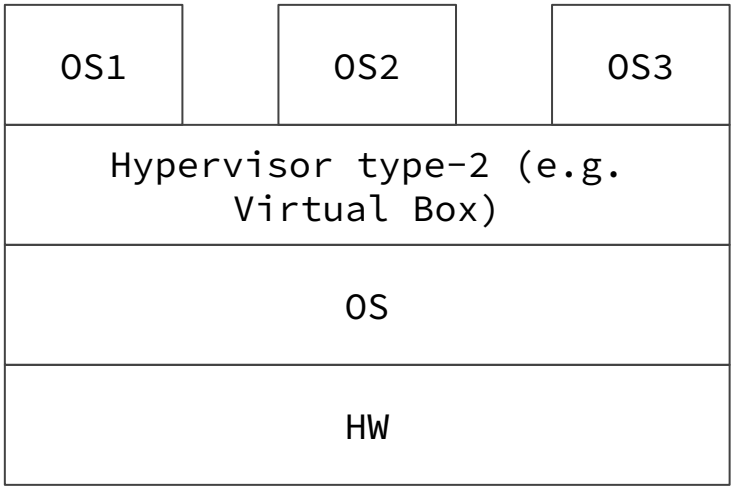


Hypervisors

Execute below OS

Isolate complete OSes from each other

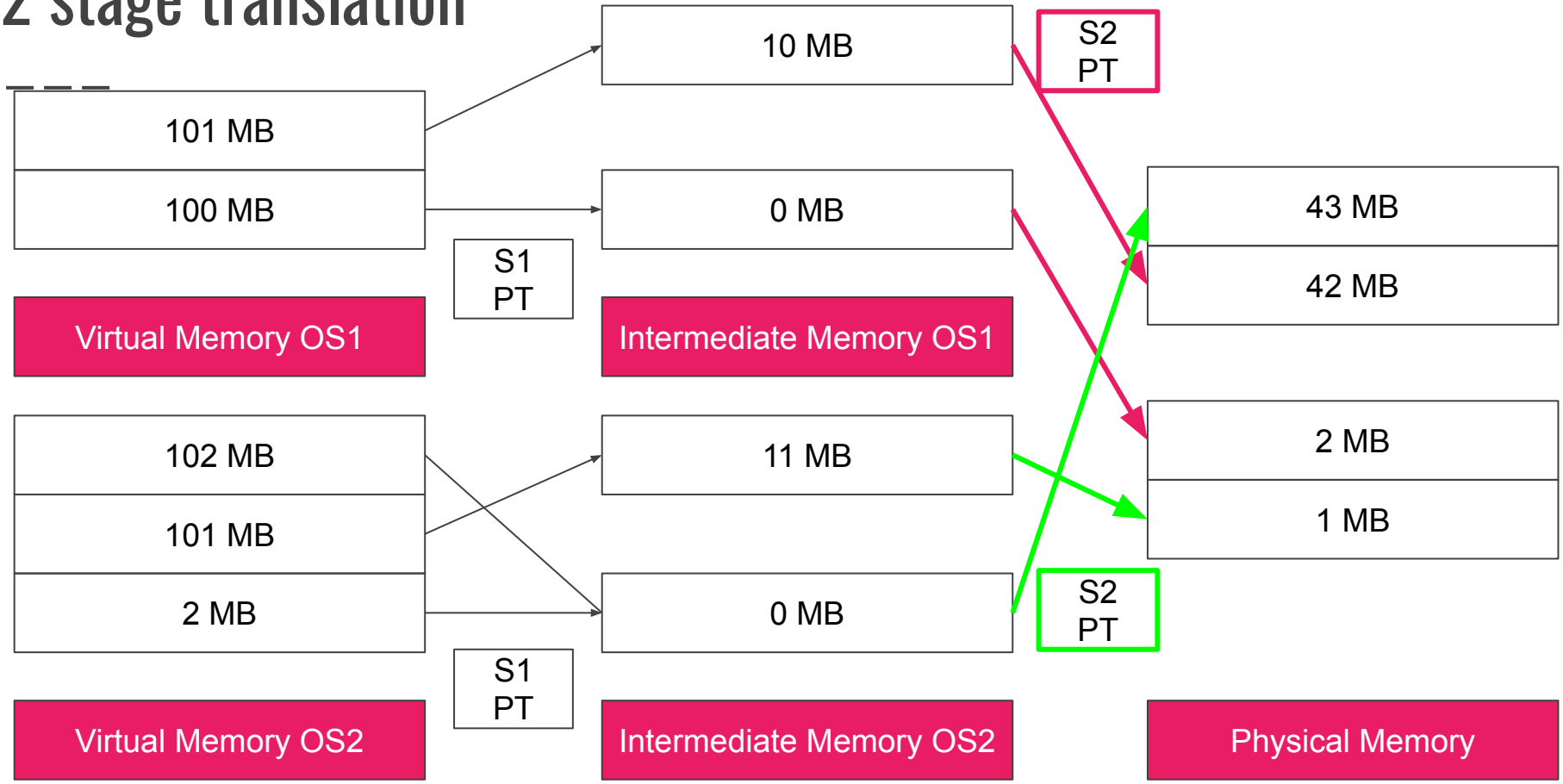
Can inspect the behavior of a (possibly) buggy OS



Hypervisors

- Use HW to isolate memory of Guests (i.e. the executed OSes)
- HW assisted virtualization
  - There is a special security domain in the CPU that is used to execute the hypervisor
  - MMU supports two stages (and corresponding page tables)
    - Virtual to Intermediate addresses
      - Configured by the OS
    - Intermediate to Physical
      - Configured by the hypervisor
  - Accesses must be granted by both stages
  - Xen >= 4.3
- VA -s1 pt-> Intermediate Address -s2 pt-> Physical Address

2 stage translation



Hypervisors

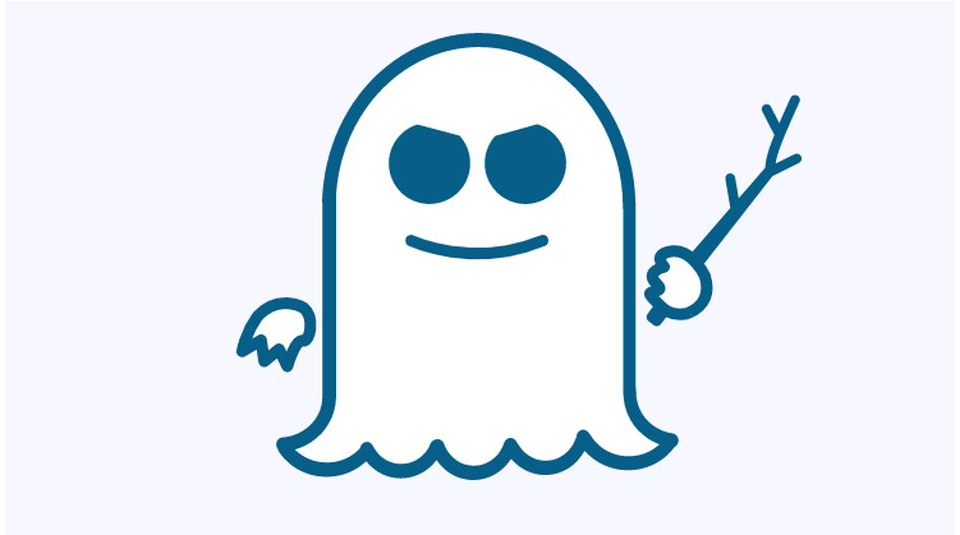
- Use HW to isolate memory of Guests (i.e. the executed OSes)
- Paravirtualizing hypervisors
  - OS kernels not allowed to use privileged operations or directly change page tables
  - OS kernel is executed in user mode
  - OS kernel invokes Hypervisor syscalls (hypercalls) to update the hardware configurations
  - Hypervisor performs only operations that do not permit to violate guest isolation
  - XEN 3.0

Trusted Execution Environments

Intel SGX, ARM trustzone, Intel TDX, ARM CCA, AMD SEV

- Memory encryption, integrity protection, and device-access protection vary by technology.
- Do not trust hypervisor (and usually owner of the physical machine)
  - Also provide some sort of memory encryption to prevent physical attacks or attacks from devices
- Resource management delegated to hypervisor (e.g., no protection against DOS)
- Provide some mechanism for remote attestation
  - Allow a trusted component to “measure” (e.g., digest of executing code and configuration)
  - Allow to establish secure channel via crypto
  - Allow to report measurements

Spectre

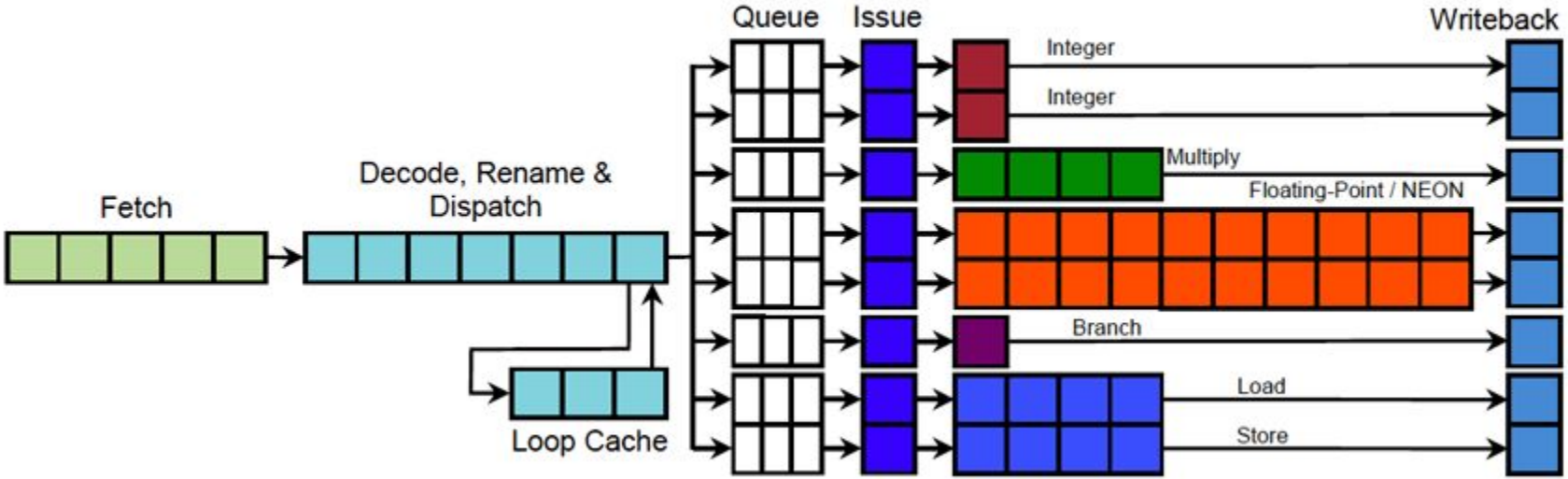


Spectre

```
int F(R0) {
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {
        R2 = LOAD #A[R0]
        R3 = LOAD #B[R2]
        STORE #output R3
    }
}

// ASSUME #A is public, #B is public and
// every i. 0 <= A[i] < size-B
Is this program non-interferent?
Is this program constant time?
```

Relative old pipeline ARM A57



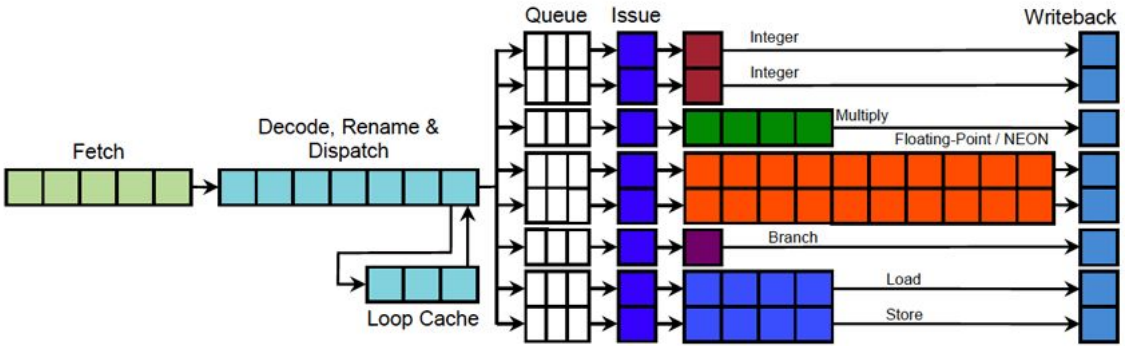
Out of order execution

```
R1 = LOAD[R0]

R2 = R3 + 3

R4 = R2

R5 = R4 + R1
```



- Memory accesses can be really slow:
  - Execute out-of order when possible

Speculative execution

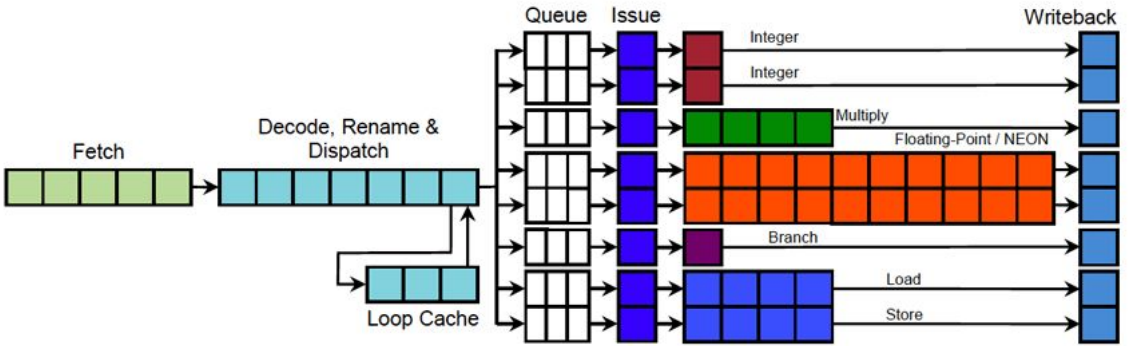
```
R1 = 0

loop: R2 = R2 * R1

R1 = R1 + 1

if R1 < 100 jmp loop

call printf R2
```



- Pipeline flushes are expensive
  - Guess R1 < 100
- Speculate executions, except memory store

Spectre

```
int F(R0) {                     // R0 = #key - #A
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {           // false
        R2 = LOAD #A[R0]        // #A+R0=#A+#key-#A     R2=key
        R3 = LOAD #B[R2]
        STORE #output R3
    }
}

// ASSUME #A is public, #B is public and
// every i. 0 <= A[i] < size-B
```

Spectre

```
int F(R0) {                    // R0 = #key - #A
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {          // false
        R2 = LOAD #A[R0]       // #A+R0=#A+#key-#A     R2=key
        R3 = LOAD #B[R2]       // Cache line accessed depends on
        STORE #output R3           R2 (key)
    }
}

// ASSUME #A is public, #B is public and
// every i. 0 <= A[i] < size-B
```

Spectre v1 attacks

- Two gadgets:
  - Reading gadgets (i.e., buffer overread)
  - Transmitting gadget (i.e., access to memory)
- Attacker poisons the branch predictor
- Attacker uses standard cache driven attack (i.e., prime/probe via access driven attack)

Spectre v1 defence: instruction serialization

```
int F(R0) {
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {
        lfence
        R2 = LOAD #A[R0]
        R3 = LOAD #B[R2]
        STORE #output R3
    }
}
```

Spectre v1 defence: instruction serialization

```
int F(R0) {
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {
        R2 = LOAD #A[R0]
        lfence?
        R3 = LOAD #B[R2]
        STORE #output R3
    }
}
```

Spectre v1 defence: index masking (SLH)

```
int F(R0) {
    R1 = LOAD #size-A

    if 0 <= R0 < R1 {
        // Create a boolean mask
        R4 = (R0 >= 0) & (R0 < R1)
        R0 = R0 & R4

        // (R0 will be 0 if out of bounds)
        R2 = LOAD #A[R0]

        R5 = (R2 >= 0) & (R2 < size-B)
        R2 = R2 & R5

        R3 = LOAD #B[R2]
        STORE #output R3
    }
}
```

Spectre v1 defence: hardware based

- Branch Target Injection Mitigations
  - Flush branch predictor state at context switch
  - Use separate branch predictors per process
- Array Indexing Bound Checks in Hardware
  - ARM Memory Tagging Extension
  - CHERRY
- Non speculative memory load

```
int F(R0) {
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {
        R2 = LOAD #A[R0]
        R3 = LOAD #B[R2]
        STORE #output R3
    }
}
```

- Non speculative cache effects
- Taint tracking

Spectre v1 defence: hardware based

- Non speculative cache effects

```
int F(R0) {
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {
        R2 = LOAD #A[R0]
        R3 = LOAD #B[R2]
        STORE #output R3
    }
}
```

- Taint tracking

Spectre v1 defence: hardware based

- Taint tracking

```
int F(R0) {
    R1 = LOAD #size-A
    if 0 <= R0 < R1 {          Branch depends on R0 and R1,
                               Both marked as untrusted
        R2 = LOAD #A[R0]       Speculative load using untrusted
                               address, R2 set as unsafe
        R3 = LOAD #B[R2]       Load with unsafe address. Stalled
        STORE #output R3
    }
}
```

Spectre v2

———

```
f:  R2 = LOAD #A[R2]
    ...
g:  R2 = LOAD #Key[R0]
    R3 = LOAD #A[R2]
    ...

R3 = &f
...
JMP R3
```

Spectre v2

———

```
f:  R2 = LOAD #A[R2]
    ...
g:  R2 = LOAD #Key[R0]  // reading gadget
    R3 = LOAD #A[R2]    // transmitting gadgets
    ...

R3 = &f
...
JMP R3                  // processor may guess target of jump
```

Spectre v2: Retpoline (from google)

```
———

f:  R2 = LOAD #A[R0]
    ...
g:  R2 = LOAD #Key[R0]
    R3 = LOAD #A[R2]
    ...

trampoline adr:
    stack.ret_ptr = adr
    ret;

trampoline(r3);
while (true) {}
```

Spectre v4

```
STORE #A[0] key
R1 = 0
R2 = 0
STORE #A[R1] 0          // A[0] = 0
R3 = LOAD #A[R2]

R4 = LOAD #B[R3]
```

Spectre v4: Store Bypass

———

```
STORE #A[0] key
R1 = 0
R2 = 0
STORE #A[R1] 0          // A[0] = 0
R3 = LOAD #A[R2]        // Processor may speculate R2 != R1
                        // and execute load before store
R4 = LOAD #B[R3]
```

Spectre v4: Store Bypass

```
STORE #A[0] key
R1 = 0
R2 = 0
STORE #A[R1] 0          // A[0] = 0
lfence
R3 = LOAD #A[R2]        // Processor may speculate R2 != R1
                        // and execute load before store
R4 = LOAD #B[R3]
```

Spectre: straight line speculation

```
int F() {
    STORE #A[0] = 0
}

F();
R2 = LOAD #A[0]      // Processor may execute in speculation
R3 = LOAD #B[R2]
```

Spectre: speculative prefetching (Apple M*)

```
for i : 0 to 100 {
    x = i % 50
    x = x & ..
    y = A[x];          // after reading 0, 1, … 49, processor
                       // may speculate index 50
    z = B[y];
    ...
}
```

Meltdown attack

Normal proceed for executing LOAD [R1]

1. Identify instruction to fetch
2. Fetch the instruction
3. Decode the instruction
4. Wait for flying instructions that are modifying R1
5. Read a = R1
6. Wait for flying instructions that are writing in a
7. Translate a from virtual to physical address pa
    a. (read page tables)
8. Check if pa is readable
    a. (read page tables)
9. Read pa
    a. If pa is in cache => hit
    b. Evict cache line and load pa.

Spectre Effects

- Web Browsers
  - crafted JavaScript code, an attacker could exploit speculative execution to read memory from the browser, including sensitive data like passwords or session tokens across different domains.
  - Google Chrome introduced Site Isolation, a feature that isolates webpages from each other in different processes.
  - Mozilla Firefox and Microsoft Edge reduced the resolution of timers to make it harder to perform fine-grained side-channel attacks.
- Enclaves (see later)
  - The SGXPectre attack demonstrated that even the highly secure Intel SGX (Software Guard Extensions)
- Linux Kernel (Meltdown and Spectre):
  - allowing unprivileged user processes to access kernel memory. This vulnerability could expose sensitive kernel data such as passwords, encryption keys, and user information.
  - Retpoline